SI405 – Análise de Sistemas de Informação II

Princípios SOLID – PART I

Prof. Ricardo Vilela

rfvilela@unicamp.br

INTRODUÇÃO

- **O que é SOLID?**

INTRODUÇÃO

➢O que é SOLID?

SOLID é um conjunto de **cinco princípios de design orientado a objetos** que ajudam a construir sistemas de software:

- mais **organizados**
- mais **flexíveis**
- mais **fáceis de manter**
- mais **fáceis de evoluir**

INTRODUÇÃO

- Os princípios **SOLID** foram propostos por **Robert C. Martin** (Uncle Bob) e são **amplamente utilizados** no desenvolvimento de **software moderno**.

- Ideia central:

Escrever código que continue funcionando bem mesmo quando o sistema cresce e precisa mudar.

INTRODUÇÃO

- O que significa SOLID?

- Acrônimo formado pelas iniciais de cinco princípios de design
  - **S**ingle Responsability Principle
  - **O**pen/Closed Principle
  - **L**iskov Substitution Principle
  - **I**nterface Segregation Principle
  - **D**ependency Inversion Principle

Importante:

SOLID **não é uma regra rígida**, mas um **conjunto de boas práticas** que ajudam a tomar decisões de design melhores.

PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

- **Coesão em Orientação a Objetos**
  - Coesão é um dos conceitos mais importantes no design orientado a objetos.
    - Uma **classe coesa** possui uma **única responsabilidade no sistema**
    - Cada classe deve representar **apenas um conceito do domínio**
    - **Exemplo:**
      - **Classe NotaFiscal → representa apenas uma nota fiscal**
      - **Classe Fatura → representa apenas uma fatura**

PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

- **Coesão em Orientação a Objetos**
  - Por que a coesão é importante?
    - classes ficam mais simples
    - código fica menor e mais organizado
    - manutenção se torna mais fácil
    - reuso de código aumenta
  - Problema comum em sistemas:
    - classes muito grandes
    - muitos métodos
    - responsabilidades misturadas
    - código difícil de entender e manter

Objetivo do bom design:

manter as classes **coesas**, com responsabilidades **claras e bem definidas**..

PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

❑Coesão em Orientação a Objetos

❑Exemplo de classe não coesa:

```
class CalculadoraDeSalario {
    public double calcula(Funcionario funcionario) {
        if(DESENVOLVEDOR.equals(funcionario.getCargo())) {
            return dezOuVintePorcento(funcionario);
        }

        if(DBA.equals(funcionario.getCargo()) ||
         TESTER.equals(funcionario.getCargo())) {
            return quinzeOuVinteCincoPorcento(funcionario);
        }

        throw new RuntimeException("funcionario invalido");
    }
}
```

PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

❑Coesão em Orientação a Objetos

❑Repare que cada uma das regras é implementada por um método privado, como o ***dezOuVintePorcento***() e o ***quinzeOuVinteCincoPorcento***(). A implementação desses métodos poderia ser como o seguinte:

```
private double dezOuVintePorcento(Funcionario funcionario) {
    if(funcionario.getSalarioBase() > 3000.0) {
        return funcionario.getSalarioBase() * 0.8;
    }
    else {
        return funcionario.getSalarioBase() * 0.9;
    }
}
```

PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

- **Qual o problema da classe?**
  - Crescimento descontrolado da classe
    - Regras diferentes para vários casos dentro da mesma classe.
    - Grande quantidade de **if** e **else**
    - Código difícil de entender
  - Código difícil de manter
    - Quanto mais regras, mais métodos e condições
    - Mudanças em uma regra pode impactar outras partes do código
  - Maior probabilidade de defeitos
  - Baixa reutilização de código
    - Métodos úteis ficam **presos dentro de uma classe grande**

PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

- **Em busca da Coesão**
  - Primeiro passo é reconhecer que a classe não é coesa
    - Toda classe que não é coesa não para de crescer nunca.
  - A classe ***CalculadoraDeSalario***, em particular, cresce indefinidamente por dois motivos:
    - Cargo novo
    - Nova Regra

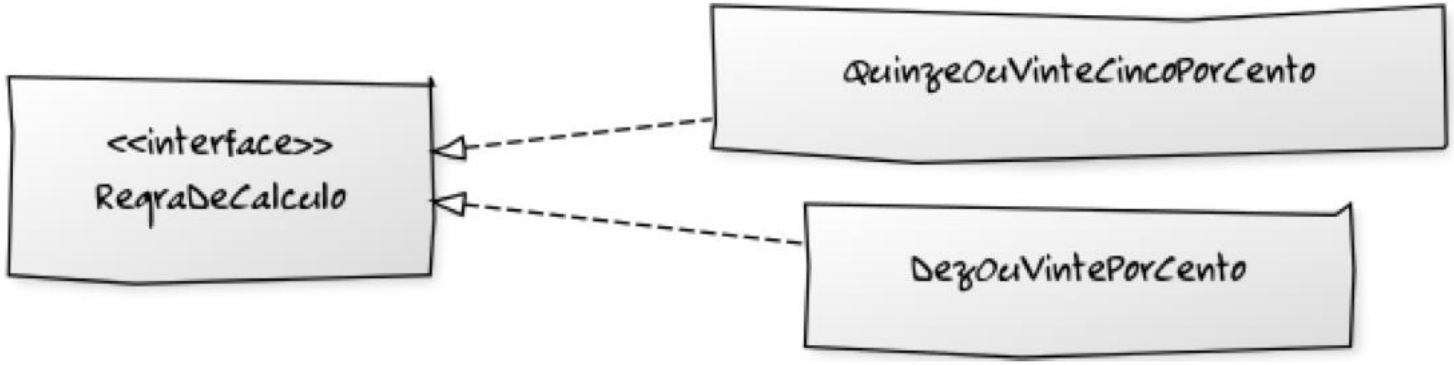
PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

- **Em busca da Coesão**
  - Vamos começar pelo segundo caso (novas regras), dada sua facilidade.
  - Ambos métodos privados possuem a mesma assinatura:
    - Recebe **funcionário** como **parâmetro** e **retornam** um **double** com salário calculado.

PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

❑Em busca da Coesão

❑Assim, uma possível solução é colocar cada regra em uma classe diferente, implementando a mesma interface.



PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

❑Em busca da Coesão

- ❑Ao isolar cada regra, uma eventual mudança em uma das regras não irá afetar a outra regra.
  - ❑COESÃO ↑

```
public interface RegraDeCalculo {
    double calcula(Funcionario f);
}
```

PRINCÍPIO DA RESPONSABILIDADE ÚNICA

❑Em busca da Coesão

```
public class DezOuVintePorCento implements RegraDeCalculo {
    public double calcula(Funcionario funcionario) {
        if(funcionario.getSalarioBase() > 3000.0) {
            return funcionario.getSalarioBase() * 0.8;
        }
        else {
            return funcionario.getSalarioBase() * 0.9;
        }
    }
}
```

```
public class QuinzeOuVinteCincoPorCento implements
RegraDeCalculo {
    public double calcula(Funcionario funcionario) {
        if(funcionario.getSalarioBase() > 2000.0) {
            return funcionario.getSalarioBase() * 0.75;
        }
        else {
            return funcionario.getSalarioBase() * 0.85;
        }
    }
}
```

PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

- **Em busca da Coesão**
  - Decisão de projeto adotada:

Cada nova regra que surgir no sistema deve ser implementada em uma classe própria.

PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

- Precisamos agora resolver o próximo problema. Este, em particular, está menos ligado à coesão, mas ainda assim é digno de atenção...
  - Considerando o exemplo apresentado, uma possível solução seria a utilização de **Enum.**
  - **Enum (enumeration) é um tipo de dado que representa um conjunto fixo e limitado de valores possíveis.**

PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

- Precisamos agora resolver o próximo problema. Este, em particular, está menos ligado à coesão, mas ainda assim é digno de atenção.

```
public enum Cargo {

    DESENVOLVEDOR(new DezOuVintePorCento()),
    DBA(new QuinzeOuVinteCincoPorCento()),
    TESTER(new QuinzeOuVinteCincoPorCento());

    private RegraDeCalculo regra;

    Cargo(RegraDeCalculo regra) {
        this.regra = regra;
    }

    public RegraDeCalculo getRegra() {
        return regra;
    }
}
```

PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

- Solução final:

```
public class Funcionario {

    private Cargo cargo;
    private double salario;

    public Funcionario(Cargo cargo, double salario) {
        this.cargo = cargo;
        this.salario = salario;
    }

    public Cargo getCargo() {
        return cargo;
    }

    public double getSalario() {
        return salario;
    }
}
```

PRINCÍPIO DA RESPONSABILIDADE ÚNICA - SRP

- Solução final:

```
class CalculadoraDeSalario {
    public double calcula(Funcionario funcionario) {
        if(DESENVOLVEDOR.equals(funcionario.getCargo())) {
            return dezOuVintePorcento(funcionario);
        }

        if(DBA.equals(funcionario.getCargo()) ||
         TESTER.equals(funcionario.getCargo())) {
            return quinzeOuVinteCincoPorcento(funcionario);
        }

        throw new RuntimeException("funcionario invalido");
    }
}
```

```
public class CalculadoraDeSalario {

    public double calcula(Funcionario funcionario) {
        RegraDeCalculo regra = funcionario.getCargo().getRegra();
        return regra.calcula(funcionario);
    }

}
```

PRINCÍPIO ABERTO-FECHADO (OCP)

- Premissa:
  - Classes devem estar **abertas para extensão** e **fechadas para modificação**
- Isso significa que devemos ser capazes de adicionar novos comportamentos ao sistema sem alterar código existente.

PRINCÍPIO ABERTO-FECHADO (OCP)

- O Problema que OCP busca resolver
  - Em sistemas reais:
    - novas regras surgem constantemente
    - novos tipos de objetos aparecem
    - novos comportamentos precisam ser adicionados
  - Se o design for ruim, isso exige modificar classes existentes o tempo todo. Ocasionando:
    - risco de bugs
    - código cada vez mais complexo

PRINCÍPIO ABERTO-FECHADO (OCP)

- Exemplo de violação: **Sistema que calcula de descontos de clientes**

```
public class CalculadoraDesconto {

    public double calcula(Cliente cliente) {

        if(cliente.getTipo() == TipoCliente.COMUM)
            return cliente.getValor() * 0.05;

        if(cliente.getTipo() == TipoCliente.PREMIUM)
            return cliente.getValor() * 0.10;

        if(cliente.getTipo() == TipoCliente.VIP)
            return cliente.getValor() * 0.20;

        return 0;
    }
}
```

PRINCÍPIO ABERTO-FECHADO (OCP)

- Exemplo de violação: **Sistema que calcula de descontos de clientes**

```
public class CalculadoraDesconto {

    public double calcula(Cliente cliente) {

        if(cliente.getTipo() == TipoCliente.COMUM)
            return cliente.getValor() * 0.05;

        if(cliente.getTipo() == TipoCliente.PREMIUM)
            return cliente.getValor() * 0.10;

        if(cliente.getTipo() == TipoCliente.VIP)
            return cliente.getValor() * 0.20;

        return 0;
    }
}
```

- **Qual o problema?**

PRINCÍPIO ABERTO-FECHADO (OCP)

- Exemplo de violação: **Sistema que calcula de descontos de clientes**
- Problema:
  - Sempre que surgir um novo tipo de cliente, **precisaremos modificar essa classe.**

```
if(cliente.getTipo() == TipoCliente.GOLD)
```

PRINCÍPIO ABERTO-FECHADO (OCP)

- Propósito Geral do OCP:
  - Em vez de modificar código existente, devemos adicionar novos comportamentos por extensão.
  - Isso é normalmente feito com:
    - Interfaces e Polimorfismo

PRINCÍPIO ABERTO-FECHADO (OCP)

- **Refatoração usando polimorfismo:**

```
public interface RegraDesconto {

    double calcula(Cliente cliente);

}
```

```
public class DescontoComum implements RegraDesconto {

    public double calcula(Cliente cliente) {
        return cliente.getValor() * 0.05;
    }
}
```

```
public class DescontoPremium implements RegraDesconto {

    public double calcula(Cliente cliente) {
        return cliente.getValor() * 0.10;
    }
}
```

PRINCÍPIO ABERTO-FECHADO (OCP)

❑Antes e Depois – CalculadoraDesconto

```
public class CalculadoraDesconto {

    public double calcula(Cliente cliente) {

        if(cliente.getTipo() == TipoCliente.COMUM)
            return cliente.getValor() * 0.05;

        if(cliente.getTipo() == TipoCliente.PREMIUM)
            return cliente.getValor() * 0.10;

        if(cliente.getTipo() == TipoCliente.VIP)
            return cliente.getValor() * 0.20;

        return 0;
    }
}
```

```
public class CalculadoraDesconto {

    public double calcula(RegraDesconto regra, Cliente cliente) {
        return regra.calcula(cliente);
    }

}
```

PRINCÍPIO ABERTO-FECHADO (OCP)

- **E se... Eu necessitar de uma nova Regra?**

PRINCÍPIO ABERTO-FECHADO (OCP)

- E se... Eu necessitar de uma nova Regra?

```
public class DescontoGold implements RegraDesconto {

    public double calcula(Cliente cliente) {
        return cliente.getValor() * 0.15;
    }
}
```

PRINCÍPIO ABERTO-FECHADO (OCP)

- O sistema agora respeita a OCP
  - Antes: adicionar regra = modificar classe
  - Depois: adicionar regra = criar nova classe
- Após refatoração, o código existente permanece sem modificações durante a adição de novas regras.

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- Objetos de uma classe derivada devem poder substituir objetos da classe base sem alterar o comportamento esperado do sistema.

Barbara Liskov

Cientista da computação estadunidense conhecida por criar o Princípio da Substituição de Liskov, por ser a primeira mulher a obter um PhD em Ciência da Computação nos Estados Unidos

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- Intuição do Princípio
- **Se B é um subtipo de A**, então:
  - Qualquer lugar que espera um A deve funcionar corretamente com um B.

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- Exemplo – Função de Retângulos

```
public void redimensionaRetangulo(Retangulo r) {
    r.setLargura(10);
    r.setAltura(5);
}
```

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- Classe Retângulo:

```
public class Retangulo {

    private int largura;
    private int altura;

    public void setLargura(int largura) {
        this.largura = largura;
    }

    public void setAltura(int altura) {
        this.altura = altura;
    }

}
```

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- Tentativa de Herança
  - É comum pensar: Um quadrado é um tipo de retângulo.
  - Então criar...

```
public class Quadrado extends Retangulo {

}
```

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- Implementação do Quadrado

```
public class Quadrado extends Retangulo {

    public void setLargura(int largura) {
        super.setLargura(largura);
        super.setAltura(largura);
    }

    public void setAltura(int altura) {
        super.setAltura(altura);
        super.setLargura(altura);
    }
}
```

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- O problema aparece
  - Agora usamos um quadrado onde se espera um retângulo.

```
Retangulo r = new Quadrado();

r.setLargura(10);
r.setAltura(5);
```

  - Resultado esperado: largura = 10 e altura = 5
  - Resultado obtido: largura = 5 e altura = 5

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- Violação do LSP
  - O comportamento mudou, ou seja, Quadrado não pode substituir Retângulo.
  - A herança quebrou o contrato da classe base.

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- Erro de Design
  - O erro não está na implementação, por si só, mas no modelo de herança.
  - Embora matematicamente Quadrado seja um tipo de Retângulo, em termos de comportamento de software, essa relação não funciona muito bem.

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- Possível Correção
  - Remover Herança e utilizar Interfaces

```
public interface Forma {

    int area();

}
```

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- Possível Correção
  - Implementações independentes:

```
public class Retangulo implements Forma {

    private int largura;
    private int altura;

    public int area() {
        return largura * altura;
    }
}
```

```
public class Quadrado implements Forma {

    private int lado;

    public int area() {
        return lado * lado;
    }
}
```

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- Possível Correção
  - Usando formas de maneira genérica:

```
public int calculaArea(Forma forma) {
    return forma.area();
}
```

```
Forma f1 = new Retangulo(10, 5);
Forma f2 = new Quadrado(5);

System.out.println(calculaArea(f1));
System.out.println(calculaArea(f2));
```

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV (LSP)

- Conclusão LSP:
  - O LSP garante que:
    - Subclasses respeitem o comportamento da classe base
    - Herança não quebre o funcionamento do sistema

SI405 – Análise de Sistemas de Informação II

Princípios SOLID – PART I

Prof. Ricardo Vilela

rfvilela@unicamp.br